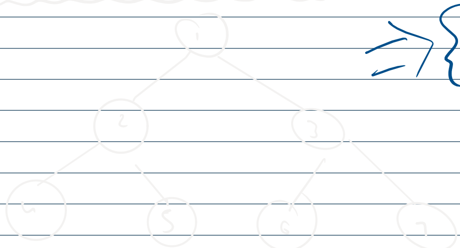
Priority Queue

It is a data structure where,
elements have priorities
removal is based on priority.

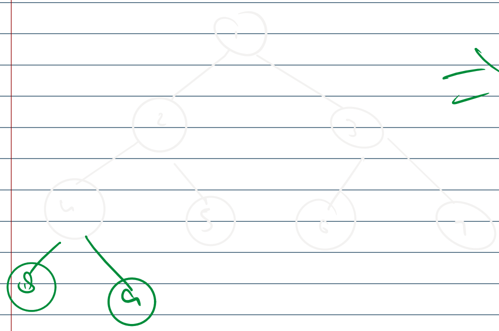
Priority Queue → Heap → Special Complete Binary Tree

min pq   max pq

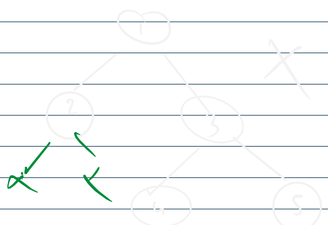
Complete Binary Tree



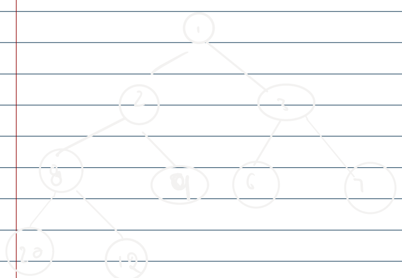
⇒ {Completely filled} ⇒ Full Binary Tree



⇒ Complete Binary Tree



Complete Binary Tree



0	1	2	3	4	5	6	7	8
1	2	3	8	9	6	7	20	12

→ arr

2 ⇒ ind = 1
→ child1 ⇒ 8 ⇒ ind = 3
→ child2 ⇒ 9 ⇒ ind = 4

8 ⇒ ind = 3
→ child1 ⇒ 20 ⇒ ind = 7
→ child2 ⇒ 12 ⇒ ind = 8

9 ⇒ ind = 4
→ parent ⇒ 2

8 ⇒ ind = 3
→ parent ⇒ 2

parent → i
child1 → 2i+1
child2 → 2i+2

child → i

parent => $\frac{(i-1)}{2}$

Heap

It should be :-
1:- CBT
2:- Heap-order property => satisfied.

Heap → min Heap
Heap → max Heap

min Heap	max Heap
parent ≤ children	parent ≥ children
root : min element	root : max element

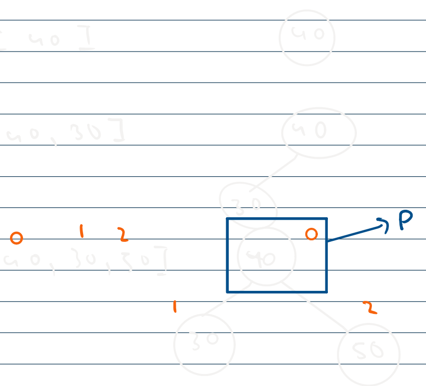
max Heap

arr → [40, 30, 50, 10, 20, 60, 70, 5, 15, 25] } Input

h : [ ]

40 h => [40]

30 h => [40, 30]



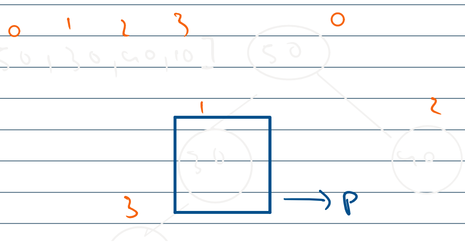
50 h => [40, 30, 50]

i = 2 => p = (i-1) / 2 = (2-1) / 2 = 0.5 = 0

50 > 40 => child > parent
↓ true
swap (50, 40)
swap (parent, child)

h => [50, 30, 40]

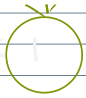
i = 0 => X

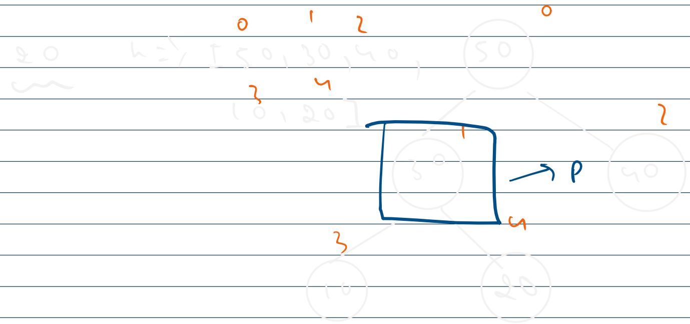


10 h => [50, 30, 40, 10]

i = 3 => p = (i-1) / 2 = (3-1) / 2 = 1

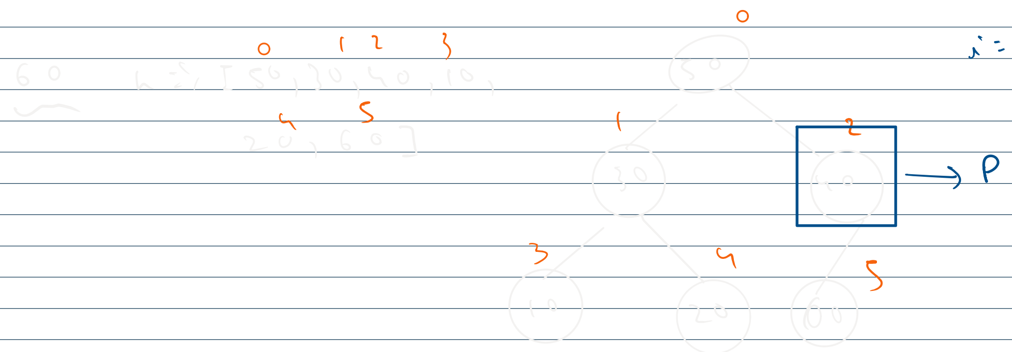
child > parent => 10 > 30 => X





$i = 4 \Rightarrow p = (i-1)/2 = 1$

child > parent ⇒ 20 > 30 ⇒ X

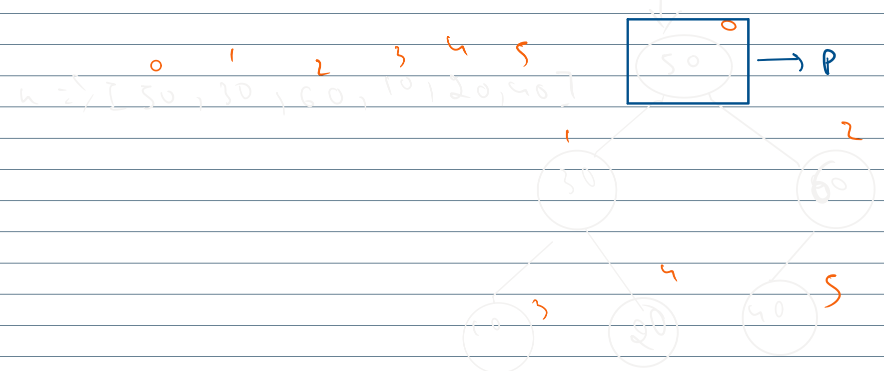


$i = 5 \Rightarrow p = (i-1)/2 = 2$

child > parent ⇒ 60 >

↓ true

swap (40, 60) ⇐ swap(p,



$i = 2 \Rightarrow p = (i-1)/2 = 0$

child > parent ⇒ 60 > 50

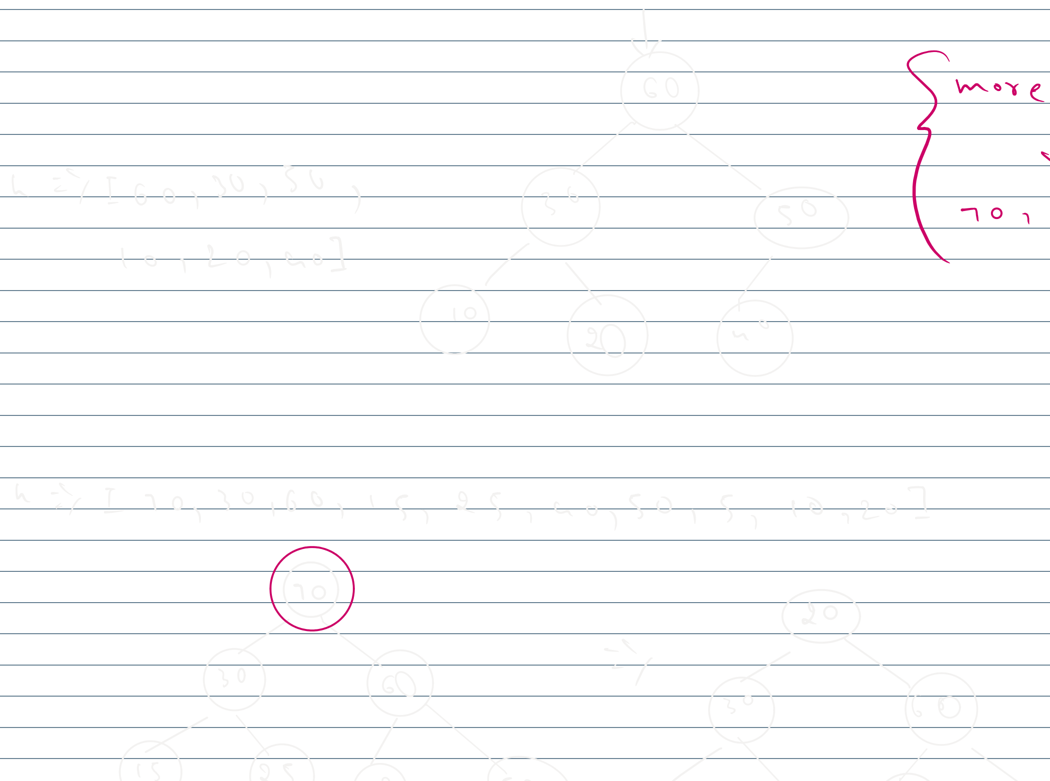
↓ true

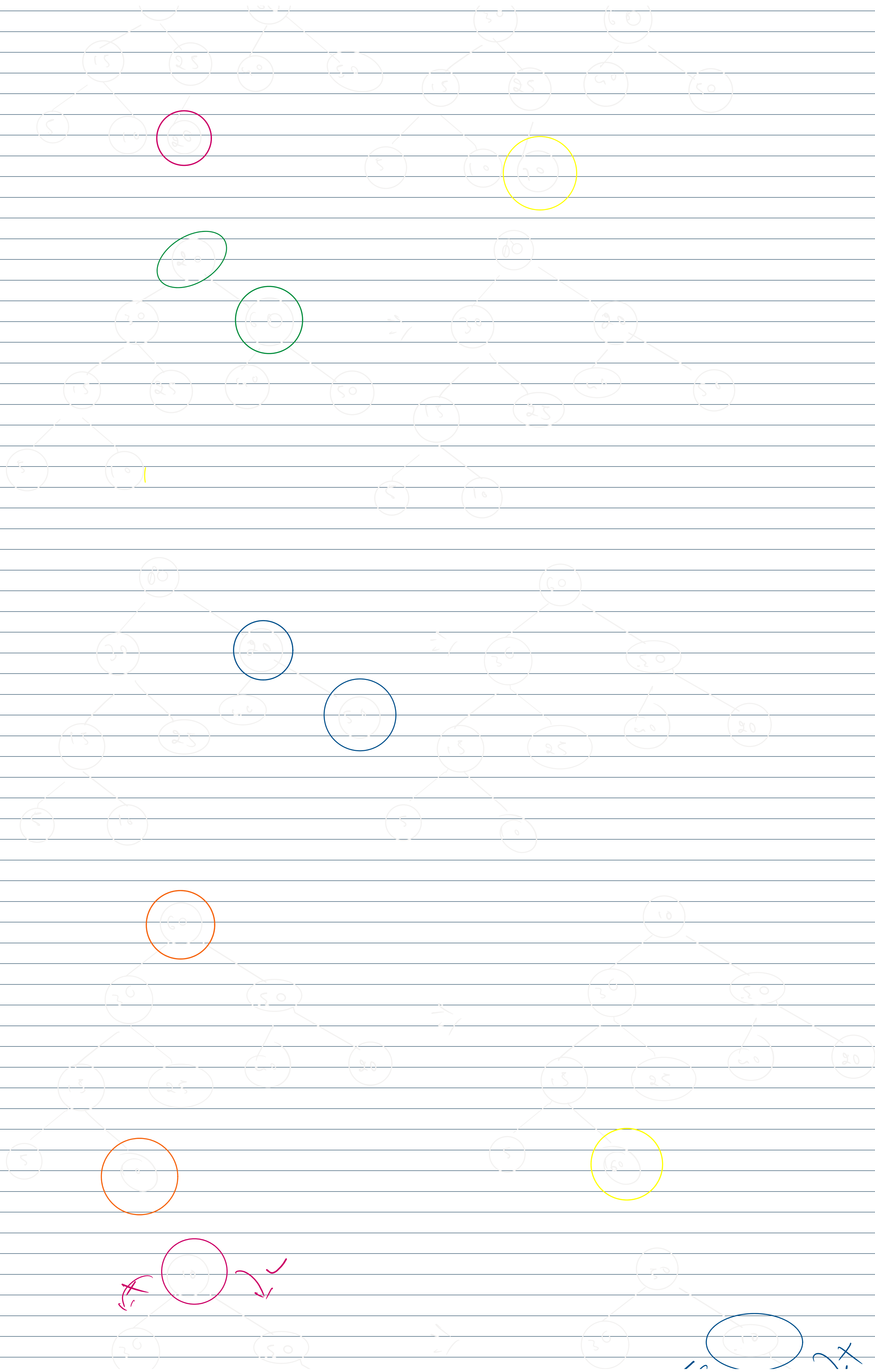
swap(50,60) ⇐ swap(p, c,

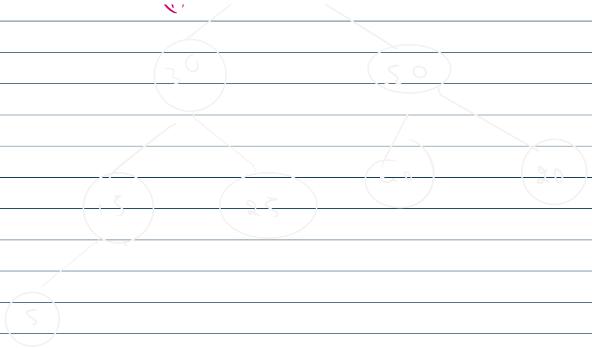
{ more to add

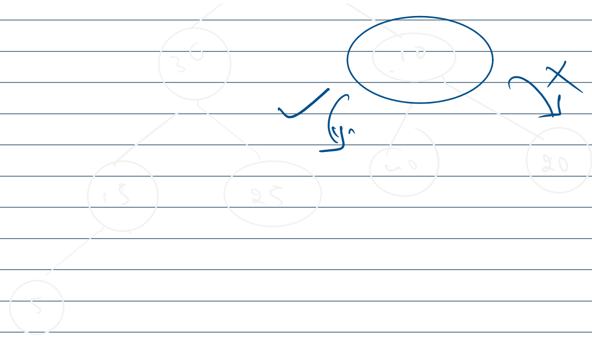
⇓

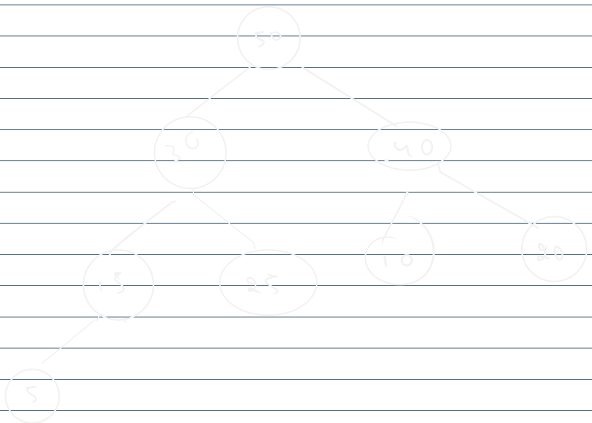
70, 5, 15, 25 }

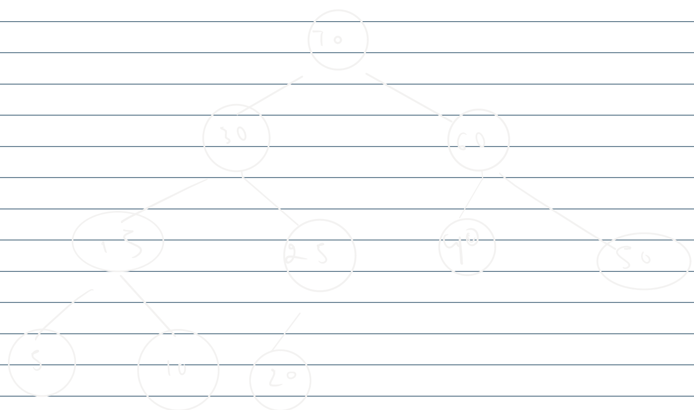












https://leetcode.com/problems/kth-largest-element-in-an-array/